CATune: Constraint-Aware DBMS Configuration Tuning

Fangping Lan
Temple University
Philadelphia, PA
fangping.lan@temple.edu

Qi Zhang
Temple University
Philadelphia, PA
qi.zhang@temple.edu

Edurad Dragut
Temple University
Philadelphia, PA
edragut@temple.edu

ABSTRACT

Praesent imperdiet, lacus nec varius placerat, est ex eleifend justo, a vulputate leo massa consectetur nunc. Donec posuere in mi ut tempus. Pellentesque sem odio, faucibus non mi in, laoreet maximus arcu. In hac habitasse platea dictumst. Nunc euismod neque eu urna accumsan, vitae vehicula metus tincidunt. Maecenas congue tortor nec varius pellentesque. Pellentesque bibendum libero ac dignissim euismod. Aliquam justo ante, pretium vel mollis sed, consectetur accumsan nibh. Nulla sit amet sollicitudin est. Etiam ullamcorper diam a sapien lacinia faucibus.

PVLDB Reference Format:
Fangping Lan, Qi Zhang, and Edurad Dragut. CATune: Constraint-Aware DBMS Configuration Tuning. PVLDB, 14(1): XXX-XXX, 2020.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at URL_TO_YOUR_ARTIFACTS.

1 INTRODUCTION

Modern Database Management Systems (DBMSs) expose hundreds of configuration knobs that govern memory allocation, concurrency control, logging behavior, and query execution strategies [20]. While this flexibility enables performance optimization across diverse workloads, it also creates a highly dimensional and heterogeneous configuration space that is notoriously difficult to navigate [29]. For instance, PostgreSQL v14.9 provides 346 knobs spanning continuous, integer, and categorical domains. In cloud environments—where workloads vary substantially and hardware configurations differ across instances—manual tuning by database administrators (DBAs) becomes increasingly infeasible [32].

To address this challenge, extensive research has proposed automated and ML-based tuning methods. These systems often achieve substantial performance improvements over manually tuned configurations. Existing approaches generally fall into two categories: (1) offline transfer-based methods that pre-train models on benchmark workloads and transfer knowledge to new deployments [28, 31, 36], and (2) online search-based methods that iteratively select configurations using search algorithms, reinforcement learning (RL) [2, 18, 25, 32], or Bayesian optimization (BO) techniques such as SMAC [6, 12, 17, 19, 28, 34, 35].

Although these methods can eventually reach well-performing configurations, they incur significant tuning costs [11, 12]. Recent studies show that state-of-the-art systems often require hundreds to thousands of iterations to converge [33], with each iteration executing time-consuming workloads. Such high cost stems from two fundamental challenges: (1) the large number of knobs requiring exploration, and (2) the broad search space associated with each knob. To address the first challenge, prior work either selects a fixed subset of knobs [2, 3, 6, 11, 32, 37], sacrificing flexibility, or performs extensive workload executions to identify important knobs [10, 28, 33], which is resource-intensive. For the second challenge, most approaches adopt vendor-provided default ranges [2, 3, 6, 12, 18, 28, 32, 34, 35]. However, these ranges are deliberately broad to preserve generality, which expands the search space unnecessarily and may even risk unstable configurations [21, 29].

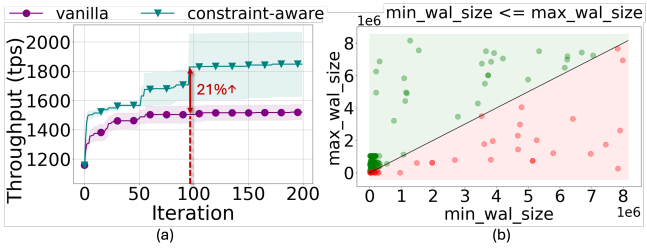

Figure 1: (a). The preliminary tuning performance improvement on the TPC-C workload using 10 knobs with/without constraint-aware tuning; (b). The valid (green) and invalid (red) samples/region during tuning based on the knob dependency constraint `min_wal_size` < `max_wal_size`.

Meanwhile, human DBAs rarely tune databases purely through blind trial-and-error. Instead, they rely on domain knowledge accumulated in DBMS manuals and community discussions. Recent knowledge-aware tuners leverage Large Language Models (LLMs) to mine such textual knowledge and guide tuning using suggested values or heuristic search region reductions [14–16, 25, 26]. While these systems successfully exploit recommended values and special cases, they primarily focus on identifying good search regions.

However, an important yet under-exploited form of knowledge lies in knob dependency constraints—relationships among knobs that define structurally invalid or suboptimal regions (e.g., ordering constraints such as $A \leq B$). These dependencies are sparsely documented in manuals and forums (see Table 1) but are not explicitly structured or encoded in existing ML-based approaches. Consequently, current tuners largely treat the configuration space as a black box and rely on trial-and-error performance feedback to implicitly learn such relationships over time. This leads to wasted evaluations exploring invalid or structurally inferior configurations. Given that each trial may involve executing complex workloads for

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

Table 1: Three examples of knob dependency constraints found in PostgreSQL V13 Documentation

Knob	Description	Constraint
superuser_reserved_connections	Determines the number of connection "slots" that are reserved for connections by PostgreSQL superusers ... The value must be less than max_connections. ...	superuser_reserved_connections < max_connections
max_parallel_workers	... Also, note that a setting for this value which is higher than max_worker_processes will have no effect, since parallel workers are taken from the pool of worker processes established by that setting.	max_parallel_workers < max_worker_processes
max_prepared_transactions	... Setting this parameter to zero (which is the default) disables the prepared-transaction feature. ... If you are using prepared transactions, you will probably want max_prepared_transactions to be at least as large as max_connections, so that every session can have a prepared transaction pending.	max_prepared_transactions ≥ max_connections when max_prepared_transactions ≠ 0

minutes or longer, such redundant exploration directly increases tuning latency and system overhead.

Our key hypothesis is that explicitly encoding knob dependency constraints into the optimization process can substantially accelerate convergence and reduce inefficient trials. Dependency constraints are particularly valuable for filtering bad regions of the search space before evaluation, thereby improving sample efficiency and robustness (Figure 1).

To realize constraint-aware tuning, we propose CATune, a system that encodes knob dependency constraints into the tuning process to proactively eliminate invalid search regions. We develop an LLM-based agent to extract and refine dependency constraints from natural language sources. We categorize constraints into three types: (1) ordering constraints (e.g., $A > B$), (2) conditional ordering constraints (e.g., $A > B$ if $A \neq 0$), and (3) unary constraints (e.g., $A \times B = c$). In this work, we focus primarily on ordering and conditional ordering constraints, as they frequently arise in practice and induce structured dependency graphs. We encode these constraints into BO-based optimizers—currently among the most effective approaches for DBMS tuning [33]. To efficiently handle multiple chained ordering constraints (e.g., $A > \cdots > B$), we design an adaptive topological sampler that maintains high sampling acceptance rates and avoids biased exploration.

Contributions. We summarize our contributions as follows.

(1) We propose CATune, a constraint-aware DBMS tuning system that encodes knob dependency constraints to eliminate invalid search regions and enhance tuning efficiency.
(2) We design an LLM-based agent to automatically extract and refine knob dependency constraints from textual knowledge sources.
(3) We introduce an adaptive topological sampler that addresses low acceptance rates and biased sampling under multiple dependency constraints.
(4) We conduct extensive experiments demonstrating that CATune improves tuning efficiency (e.g., reducing invalid trials and accelerating convergence) and achieves superior final performance.

2 PRELIMINARIES

In this section, we formalize the constraint-aware DBMS knob configuration problem and discuss existing methods for tuning. Then, we identify some limitations of existing tuners and opportunities for improvements.

2.1 Problem Definition

Classical DBMS Tuning Problem. DBMS expose tens to hundreds of configurable knobs to control their runtime behaviours [29]. The DBMS knob tuning problem is to select an appropriate value for each knob to optimize the DBMS performance (e.g., throughput or latency) on a certain workload (e.g., a workload is a set of SQL statements). Formally, given a set of configurable knobs $\theta_1, \ldots, \theta_n$ along with their domains $\Theta_1, \ldots, \Theta_n$, the configuration space is defined as $\boldsymbol{\Theta} = \Theta_1 \times \ldots \times \Theta_n$. We want to find a configuration $\boldsymbol{\theta}^* \in \Theta$ to maximize DBMS performance f:

$$\boldsymbol{\theta}^* = \arg\max_{\theta \in \boldsymbol{\Theta}} f(\theta) \tag{1}$$

Constraint-Aware DBMS Tuning Problem. DBMS configuration spaces are often subject to deterministic knob dependency constraints documented in manuals and online guidelines. These constraints define a feasible subset of the nominal configuration space. Let $C = \{c_1, \ldots, c_m\}$ denote a set of knob dependency constraints, where each constraint c_j defines a feasibility rule over configurations $\boldsymbol{\theta}$. We consider three types of knob dependency constraints: (1) *Ordering constraints*: $\theta_i \leq \theta_j$; (2) *Conditional ordering constraints:* if $h(\theta_j) = \texttt{True}$, then $\theta_i \leq \theta_j$, where $h(\theta_j)$ is simply an (in) equality predicate; and (3) *Unary constraints:* $g(\boldsymbol{\theta}) = \kappa$, where $g : \boldsymbol{\Theta} \to \mathbb{R}$ is a deterministic function over one or more knobs, and κ is a predefined constant. The feasible configuration space induced by these constraints is defined as

$$\boldsymbol{\Theta}_C = \left\{ \boldsymbol{\theta} \in \boldsymbol{\Theta} \;\middle|\; c_j(\boldsymbol{\theta}) = \texttt{True},\ \forall c_j \in C \right\}. \tag{2}$$

The constraint-aware tuning problem is therefore formulated as

$$\boldsymbol{\theta}^* = \arg\max_{\boldsymbol{\theta} \in \boldsymbol{\Theta}_C} f(\boldsymbol{\theta}), \tag{3}$$

Compared to classical tuning, which optimizes over the full space $\boldsymbol{\Theta}$, constraint-aware tuning restricts optimization to the feasible

manifold $\Theta_C \subseteq \Theta$, thereby eliminating structurally invalid configurations and reducing unnecessary exploration.

2.2 Related Work

2.2.1 Database Configuration Tuning. Three kinds of approaches are proposed for database configuration tuning: heuristic-based, Bayesian Optimization (BO)-based [24] and Reinforcement Learning (RL)-based [9].

Heuristic-based. Rule-based methods rely on manually created rules to explore the space in a predefined way [4, 13]. Search-based methods explore the space according to several heuristics (e.g., avoid to revisit explored regions and explore nearby regions to improve the current optimum) [1, 37].

BO-based. BO-based methods [3, 6, 12, 28, 34, 35] follow the generic BO framework to search for an optimal configuration: (1) fitting a probabilistic surrogate model to map the relation between knob configuration and DBMS performance, (2) selecting the next configuration that maximizes acquisition function.

RL-based. RL-based methods explore search space with a trial-and-error strategy. The essence is to balance between exploring unexplored space and exploiting known regions, which is achieved by the interactions between an agent (e.g., neural network) and the target environment (e.g., DBMS) [2, 18, 27, 30, 32].

2.2.2 Constrained Bayesian Optimization. Constrained Bayesian Optimization (CBO) extends Bayesian optimization to problems with unknown or partially observed constraints [7, 8, 22, 23]. Prior CBO methods typically model constraints as black-box functions using probabilistic surrogates (e.g., Gaussian processes) and incorporate feasibility through acquisition functions such as Expected Improvement with Constraints or Probability of Feasibility. While effective in general settings, these approaches often sample infeasible configurations during early exploration and must learn constraint boundaries from data, which can be inefficient in system-level optimization. In contrast, we focus on a practically important class of *deterministic knob dependency constraints* in database configuration spaces. Rather than learning feasibility probabilistically, we explicitly encode dependency constraints into the configuration space so that all sampled configurations are valid by construction. This differs fundamentally from classical CBO and is better suited for database tuning, where constraint violations may cause instability or system crashes [21, 29].

2.3 Motivation

M1: Tuning budget waste significantly increases the cost of machine learning (ML)-based DBMS tuning. State-of-the-art ML-based tuning methods typically require hundreds of iterations to converge to a high-quality DBMS configuration, resulting in substantial tuning overhead. However, a non-trivial portion of this budget is effectively wasted. In practice, many sampled configurations are either invalid or uninformative. For example, in our preliminary experiments, approximately 35% of 200 tuning iterations led to DBMS crashes. Additionally, numerous configurations violate knob dependency constraints (e.g., `superuser_reserved_connections` must be strictly smaller than `max_connections`), producing executions that are either rejected or semantically meaningless. Such budget waste not only increases the operational cost of tuning but also slows down model convergence. Since ML-based optimizers rely on observed performance signals to update surrogate models, invalid or low-quality configurations contribute little useful information. Consequently, a substantially larger number of observations is required before the model can form confident predictions, particularly in high-dimensional and constraint-heavy configuration spaces.

M2: Domain knowledge helps, but is not fully exploited. Extensive DBMS tuning knowledge has accumulated over decades, primarily documented in natural language through manuals, best practices, and expert guidelines. Such knowledge is invaluable for reducing search complexity and guiding configuration exploration. Prior work has made meaningful efforts to incorporate portions of this knowledge into automated tuning frameworks, including suggested values to try [25, 26], range constraints, and special-value handling [14–16]. However, a critical aspect remains underexplored: *explicit dependency relationships between knobs.* These dependencies substantially influence tuning effectiveness, determining both the number of valid iterations (i.e., the configuration satisfies the knob dependencies) and the achievable configuration quality (i.e., better final performance). In existing ML-based tuning methods, such dependencies are not explicitly modeled but are expected to be implicitly learned from observed samples. This implicit learning is data-hungry and unreliable, particularly in high-dimensional and constraint-heavy spaces, and there is no guarantee that the learned surrogate model faithfully captures these structural relationships. As shown in Figure 1, modeling knob dependency constraints directly improves tuning efficiency. In Figure 1(a), incorporating dependencies yields approximately 21% higher throughput within the first 100 iterations, demonstrating faster convergence. Figure 1(b) further illustrates that constraints such as `min_wal_size` $\leq$ `max_wal_size` eliminate nearly 50% of the nominal search region, and enforcing it could reduce invalid samples by 19.5% during tuning. These results highlight that dependency constraints substantially shrink the effective search space and reduce wasted trials. Despite their practical importance, such dependency knowledge is often implicit, scattered across documentation, and largely accessible only to experienced DBAs. Existing automated tuning approaches either overlook these dependencies or encode them in limited, static forms, leaving substantial performance gains unrealized.

M3: LLMs offer new opportunities for dependency extraction, but remain insufficient. Domain knowledge embedded in DBMS manuals, technical documentation, and forum discussions often contains rich descriptions of configuration dependencies. Traditionally, such knowledge has been difficult for machines to leverage due to the unstructured and natural-language format in which it is expressed. With the advent of large language models (LLMs), it becomes possible to automatically extract these dependency relationships and transform them into machine-readable representations (e.g., structured rules, JSON specifications, or relational tables). However, this process is still far from trivial. First, dependency information is often scattered across lengthy documents and informal discussions, requiring complex processing steps such as ingestion, filtering, normalization, and integration of heterogeneous sources. Second, LLM

outputs can be brittle—small prompt variations may lead to significantly different extracted rules. Moreover, hallucination remains a concern: LLMs may generate plausible-looking but incorrect dependencies, which can mislead downstream tuning processes. As a result, while LLMs substantially lower the barrier to mining manual knowledge, reliable and accurate dependency extraction still requires careful validation and systematic design.

M4: The absence of a systematic mechanism for injecting constraints into DB tuning. Current optimization algorithms, including Bayesian Optimization (BO) and Reinforcement Learning (RL), are primarily designed for black-box search and only natively support basic range constraints [14–16]. They do not provide direct support for incorporating richer knowledge, such as inter-knob dependencies or conditional rules, without substantial modifications to their standard workflows. As a result, much of the available constraint information remains unused. While simple heuristics (e.g., manually restricting values or prioritizing suggested configurations) can be applied, these ad hoc approaches are neither scalable nor theoretically grounded. This limitation is particularly critical in knob tuning, where the search space is combinatorial and the problem is NP-hard. Without a dedicated constraint-aware framework, optimization remains inefficient and prone to exploring invalid or suboptimal configurations.

3 KNOB DEPENDENCY CONSTRAINT EXTRACTION

(1) Assume we have a list of knobs, aiming to have an automatic system to extract the relationship between knobs.
   - Explicit relationships: "A should be larger than B" $\rightarrow$ A > B
   - Implicit relationships:
     (a) from knob names: min_A, max_A, $\rightarrow$ min_A < max_A;
     (b) from statement: "when set A to B or larger, it may impact performance." $\rightarrow$ A < B;
     (c) from resource-related knobs: knob A sets the buffer of index, which is related to the RAM size of the hardware, then A is implicitly bounded by the hardware resource, i.e., RAM size.
   - Conditional relationships: "if A is enabled, then A should be greater than B. (assume A = 0 is disabled)" $\rightarrow$ A > B if A $\neq$ 0. It involves what are called *hybrid knobs* [11] that the knobs contain two types of values: numerical values (having a natural order) and special values (e.g., -1, 0) that function differently from numerical values that inevitably break this natural order (e.g., disable some feature). Here, we only consider the hybrid knobs whose numerical values have ordering constraints with other knobs.
   - Others: A $\times$ B < C. C might not be knobs (Currently, I have not found the case that C is a knob).

(2) Types of Constraints
   - Ordering Constraints: A > B, B > C.
   - Conditional Ordering Constraints: A > B if A $\neq$ 0
   - boundary Constraints: A < RAM
   - Unary Constraints: A $\times$ B = C

(3) Compare the results with manually extracted relationships
   - How many relations are not aligned with manually extracted relationships
   - The types of misalignment: missing, ones due to hallucinated context, ones that incorrectly interpret relationships, etc.

3.1 Constraint Extraction Workflow

3.2 Constraint Analysis

We classify PostgreSQL configuration constraints into hard safety rules, whose violation can cause startup failure, feature disablement, or transactional errors, and soft tuning rules, which guide performance optimization without affecting correctness. Hard rules primarily enforce structural resource hierarchies and transactional capacity limits, while soft rules align memory, vacuum behavior, and query planning strategies with workload characteristics such as write intensity or join complexity.

Definition. *Hard Constraints*: Violating these rules can lead to database startup failure, runtime errors, disabled features, or severe correctness risks. They must always be enforced.

Definition. *Soft Constraints*: Violations do not crash PostgreSQL but may cause performance degradation, instability, or pathological planner/runtime behavior.

Definition. **Resource-bounded Constraints**: Resource-bounded Constraints are configuration constraints that restrict the admissible range of tuning knobs based on the physical limits of system resources—such as main memory (RAM), CPU cores, and storage capacity or I/O bandwidth—to ensure that the database system does not oversubscribe hardware resources or violate fundamental execution assumptions.

4 CONSTRAINT-AWARE TUNING

In this section, we present a new approach that guides the BO in database configuration tuning, which bypasses the invalid configurations, yet realizes the benefits of constraint-aware tuning.

4.1 Constraint Integration

Ordering Constraint. We incorporate ordering constraints by redefining the feasible configuration domain as a structured subspace of the nominal configuration space. Rather than optimizing over the full Cartesian product of knob domains, the optimizer operates over a feasible manifold induced by deterministic dependency constraints. Formally, the admissible search space is restricted to configurations that satisfy all ordering constraints, thereby eliminating infeasible regions *a priori*. Both exploratory sampling and surrogate-guided candidate selection are confined to this structured subspace. As a result, the optimization process no longer wastes evaluations on invalid configurations, transforming the tuning problem into constrained optimization over a reduced, topology-aware configuration manifold. This structural restriction improves sample efficiency by reducing the effective search volume while preserving all feasible optima.

Hybrid Knobs and Conditional Ordering Constraint. Certain DBMS knobs are *hybrid* [11], meaning that their domains contain both

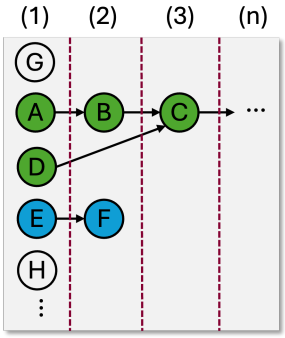

Figure 2: The topological graph induced by ordering constraints. Nodes correspond to knobs, and a directed edge $A \rightarrow B$ represents the constraint $A > B$.

numerical values and special categorical values. These special values often enable or disable internal functionalities (e.g., switching algorithms or activating components), and their semantics differ fundamentally from ordinary numerical settings. When a hybrid knob is set to such a special value, it may drastically change system behavior and have a significant impact on performance [11, 15].

Conditional ordering constraints frequently arise in conjunction with hybrid knobs. For example, an ordering constraint may only apply when a hybrid knob takes a specific value. However, in practice, optimizers may rarely sample these special values because their probability under uniform or model-based sampling is extremely low [11]. Moreover, performance models may learn that decreasing the knob value toward zero improves performance, further reducing the likelihood of selecting the special value [21]. As a result, conditional ordering constraints tied to these values are seldom activated during optimization.

To address this issue, inspired by GPTuner [14], we introduce a *constraint activation rule*. Each rule consists of (i) a control knob with a binary state, (ii) a designated activation value, and (iii) an ordering constraint that becomes effective when the control knob equals the activation value. When the control knob is assigned the activation value, the associated ordering constraint is enforced; otherwise, it remains inactive. This mechanism explicitly increases the sampling probability of special hybrid-knob values and ensures that the corresponding conditional ordering constraints are meaningfully explored during tuning.

4.2 Adaptive Topological Sampling

4.2.1 Low Sampling Acceptance Rate. The naive constraint integration relies on rejection sampling: we sample and examine a configuration, and if the sample is not satisfied by all constraints, it is rejected, and then sample the next configuration until we find a configuration that satisfies all constraints. However, when the chain of ordering constraints becomes longer (e.g., constraints between knobs, A, B, C, and more in Figure 2), the issue of low sampling acceptance rate arises, especially when the search space occupies only a vanishingly small fraction of the Cartesian product of knob domains.

Naive constraint integration uses forbidden logics to avoid invalid search space. In constraint-based configuration spaces that rely on rejection sampling, low acceptance probability typically arises when the feasible region defined by constraints occupies only a vanishingly small fraction of the Cartesian product of parameter domains. This often occurs when semantic relationships between parameters are enforced solely through forbidden clauses while their sampling ranges remain unconstrained or poorly aligned. In particular, inequalities that compare parameters with vastly different domain scales (e.g., a wide-range integer parameter constrained to be smaller than a narrow-range parameter) drastically reduce the probability that independently sampled values satisfy all constraints. As the number of such constraints increases, the joint acceptance probability decays multiplicatively, leading to exponential growth in the expected number of samples required to obtain a single valid configuration. In these settings, rejection sampling becomes impractical despite the correctness of the constraints themselves.

Case Study. In a PostgreSQL tuning configuration space with 46 hyperparameters and over 20 forbidden clauses, several constraints enforce that parameters drawn from very large ranges (e.g., [0, 262,143]) must be strictly smaller than parameters drawn from much narrower ranges (e.g., [24, 100], suggested ranges by LLM [14]). For example, constraints such as superuser_reserved_connections < max_connections and max_wal_senders < max_connections each admit valid samples with probability on the order of 10^{-4}. When combined, these constraints alone reduce the acceptance probability to approximately 10^{-8}, even before accounting for additional inequalities among worker and memory parameters. Consequently, a rejection-based sampler is expected to generate tens of millions of random candidates to obtain a single valid configuration, causing the sampling process to appear stalled while repeatedly executing low-level random number generation.

4.2.2 Biased Sampling Data. We observe that naïve topology-aware sampling induces a severe bias in the feasible configuration space, even at the random exploration stage. This geometric distortion leads to reduced coverage of valid regions and causes Bayesian optimization to prematurely collapse to a narrow subset of the space, resulting in inferior performance compared to unconstrained optimization.

Figure 3 presents a diagnostic analysis of the constraint pair `vacuum_freeze_min_age < autovacuum_freeze_max_age`, visualizing sampled configurations produced by SMAC under both unconstrained (baseline) and topology-aware (topo) settings. To disentangle the effects of exploration and optimization, we further partition the samples into random exploration and model-proposed configurations.

Observation 1: Baseline Sampling Preserves Exploration Geometry. Under the baseline setting, random samples uniformly cover the rectangular configuration space, while model-proposed samples cluster adaptively without collapsing exploration. Although baseline sampling produces infeasible configurations that violate the ordering constraint, it preserves high-entropy coverage across both dimensions, providing the surrogate model with rich contrastive signals.

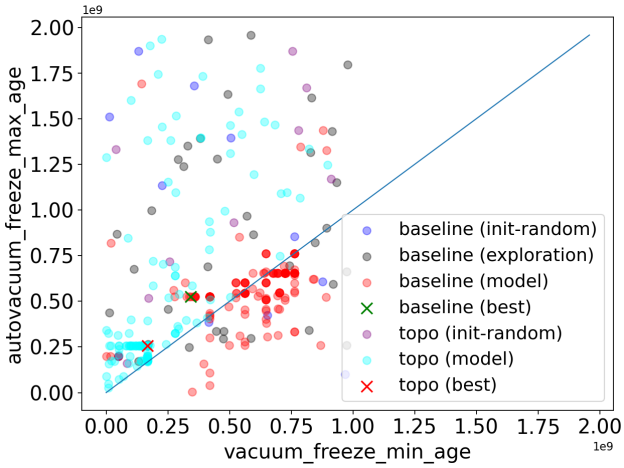

Figure 3: The biased sampling points of knon dependency constrain vacuum_freeze_min_age < autovacuum_freeze_max_age because the naive constraint-aware sampling changes the search space distribution.

Observation 2: Topology-Aware Random Sampling Is Already Biased. In contrast, topology-aware random samples exhibit a strong concentration toward small values of vacuum_freeze_min_age and the lower portion of the feasible region. Large valid regions near the upper-right of the feasible triangle are sparsely explored or entirely absent. This bias appears before any model-based optimization, indicating that the constraint-handling mechanism itself reshapes the sampling distribution.

Observation 3: Model-Based Optimization Amplifies Sampling Bias. Model-proposed samples under topology-aware sampling further collapse into a narrow region near the lower-left corner of the feasible space. Rather than discovering diverse high-performing configurations, Bayesian optimization over-exploits the already-biased region it has observed, resulting in premature convergence and reduced performance.

Root Cause: Feasible-Space Geometry Distortion. This failure arises from sequential conditional sampling of ordered parameters, which implicitly assigns higher probability mass to configurations with small root values and progressively shrinks the feasible support as the root parameter increases. Although the constraint is satisfied, the resulting distribution is not uniform over the feasible region, violating the implicit geometric assumptions of surrogate models used in Bayesian optimization.

Implication. This failure case demonstrates that correctness-preserving constraint enforcement is insufficient for effective optimization. Topology-aware sampling that distorts feasible-space geometry can significantly degrade exploration entropy, leading to worse optimization performance than unconstrained baselines despite higher feasibility rates. Constraint handling must therefore preserve not only feasibility, but also the geometric and statistical properties required for effective surrogate learning.

4.3 Topological Sampling

Algorithm 1: Build a Topology Graph from Constraints

Input: Constraint list $\mathcal{R} = \{(a, \text{op}, b)\}$ where op $\in \{<, \leq, >, \geq\}$.
Output: Directed graph $G = (V, E)$ with edge labels storing op.
$V \leftarrow \emptyset, E \leftarrow \emptyset$
foreach $(a, \text{op}, b) \in \mathcal{R}$ **do**
  $V \leftarrow V \cup \{a, b\}$
  if op $\in \{<, \leq\}$ **then**
    $E \leftarrow E \cup \{(b \rightarrow a, \text{op})\}$
  else
    $E \leftarrow E \cup \{(a \rightarrow b, \text{op})\}$
if *G contains a directed cycle* **then**
  return FAIL `// constraints are mutually inconsistent for topological sampling`
return G

Algorithm 2: Topological Sampler

Input: Configuration space C with d knobs; topology graph $G = (V, E)$; activation rules $\mathcal{A}$; number of samples N.
Output: A set of valid configurations $\mathcal{S} = \{\mathbf{c}_1, \ldots, \mathbf{c}_N\}$.
Compute a topological order $\pi \leftarrow \text{TopoSort}(G)$
Let $\mathcal{R} \leftarrow \{v \in V \mid \text{indegree}(v) = 0\} \cup \{hp \in C \mid hp \notin V\}$
`// root knobs and independent knobs`
Initialize $\mathcal{S} \leftarrow \emptyset$ `// sample set`
`// (1) Sample roots`
Sample $\mathbf{U} \in [0, 1]^{N \times |\mathcal{R}|}$ for $\mathcal{R}$ using LHS;
for $i = 1 \ldots N$ **do**
  Initiate an empty configuration $\mathbf{c}_i$;
  for $j = 1 \ldots |\mathcal{R}|$ **do**
    $\mathbf{c}_i[hp_j] \leftarrow \text{convert}(u_{ij}, hp_j)$
`// (2) Sample remaining knobs in topological order`
for $i = 1$ **to** N **do**
  foreach $x \in \pi \setminus \mathcal{R}$ **do**
    if $\neg\text{Active}(x, \mathbf{c}_i, \mathcal{A})$ **then**
      continue
    Initialize feasible space $I_x = \text{def_space}(x) = [l_x, h_x]$
    foreach *incoming edge* $(p \rightarrow x, \text{op}) \in E$ **do**
      $h_x \leftarrow \min(h_x, \mathbf{c}_i[p])$ `// Tighten` I_x
    Sample $\mathbf{c}_i[x]$ uniformly from I_x
  $S \leftarrow S \cup \{\mathbf{c}_i\}$
return $\mathcal{S}$

Motivation.

Bias for Child Nodes. TopoSampler works like: 1. sample parent (root) uniformly 2. sample child uniformly from the restricted interval (e.g., child ∈ [low, parent])

That satisfies constraints, but it does not sample uniformly from the feasible set child ≤ parent.

Why this matters more in the "suggested small range" case In a huge default space, this bias can be beneficial because it implicitly

Algorithm 3: Standard Bayesian Optimization for Database Tuning. BO for DB tuning with Topological Sampler. Blue underlined text highlights differences to the original BO.

Input: Database system DB; configuration space $\mathcal{C}$ with constraints $\mathcal{R}$ and activation rules $\mathcal{A}$; performance metric $f(\cdot)$ (e.g., throughput / latency); objective type obj $\in$ {throughput, latency}; initial design size n_0; evaluation budget T.

Output: Best configuration $\mathbf{c}^*$.

Let $\mathcal{D} \leftarrow \emptyset$; `// Tuning trajectories`

Let $G \leftarrow$ `BuildTopoGraph(`$\mathcal{R}$`)`; `// Build topology graph, see Algorithm 1`

$\mathcal{C}_{init} \in \{\mathbf{c}_1 \dots \mathbf{c}_{n_0}\} \leftarrow$ `TopoSampler(`$\mathcal{C}, G, \mathcal{A}, n_0$`)`; `// See Algorithm 2`

for $\mathbf{c} \in \mathcal{C}_{init}$ **do**
  Apply $\mathbf{c}$ to DB and observe $y \leftarrow f(\mathbf{c})$;
  $\mathcal{D} \leftarrow \mathcal{D} \cup \{(\mathbf{c}, y)\}$;

for $t = n_0 + 1 \dots T$ **do**
  $\mathcal{M} \leftarrow$ `Train(`$\mathcal{D}$`)`; `// Train a surrogate model`
  With probability p, select $\mathbf{c}_t$ for exploration; otherwise select $\mathbf{c}_t$ for exploitation **if** rand$(0, 1) < p$ **then**
    $\mathbf{c}_t \leftarrow$ `TopoSampler(`$\mathcal{C}, G, \mathcal{A}, 1$`)`; `// exploration, maximizes model uncertainty in` $\mathcal{C}$
  else
    $\mathbf{c}_t \leftarrow$ `SuggestNext(`$\mathcal{M}, \mathcal{C}, \mathcal{D}$`)`; `// exploitation, maximizes high feasibility area in` $\mathcal{C}$
  Apply $\mathbf{c}_t$ to DB and observe $y_t \leftarrow f(\mathbf{c}_t)$;
  $\mathcal{D} \leftarrow \mathcal{D} \cup \{(\mathbf{c}_t, y_t)\}$

if obj = throughput **then**
  $\mathbf{c}^* \leftarrow \arg\max_{(\mathbf{c},y)\in\mathcal{D}} y$

else
  $\mathbf{c}^* \leftarrow \arg\min_{(\mathbf{c},y)\in\mathcal{D}} y$

return $\mathbf{c}^*$;

regularizes away extreme/unreasonable values and avoids wasting time near bad regions. In a tight "already good" range, the optimum is often near a boundary (e.g., "use as much memory as allowed", "set workers close to limit", etc.). But TopoSampler's conditional uniform tends to produce children spread across the entire allowed interval, which makes it unlikely to hit "near-parent / near-ceiling" values.

Concrete example with shared_buffers $\leq$ max_wal_size: If max_wal_size is sampled around the middle of its (tight) suggested range, TopoSampler makes shared_buffers uniform in [low, max_wal_size], which systematically pulls shared_buffers downward. If best performance needs shared_buffers $\approx$ max_wal_size (common in tuning "pack it to the ceiling"), TopoSampler under-samples that region.

4.4 Biasing Constraints

Biasing constraints according to workload characteristics, such as read-heavy, write-heavy, write-ready-mixed.

To avoid accidentally triggering GEQO due to join collapsing This ensures: • The planner never collapses joins into a list large enough to hit the GEQO threshold because of collapsing alone • GEQO only activates for genuinely large join graphs This is why PostgreSQL defaults are aligned this way. join_collapse_limit < geqo_threshold

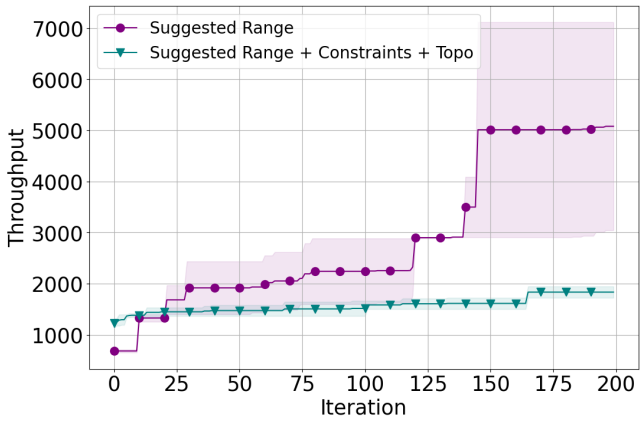

Figure 4: The tuning performance is reduced by topological sampling due to the disuniformity over the feasible region.

5 EVALUATION

In this section, we evaluate CATune on various workloads and show that it can improve the time taken for tuning PostgreSQL on n different workloads by $n\times$ up to $m\times$ when using SMAC.

Research Questions. **RQ1: Does injecting manual knob dependency rules improve DB configuration tuning performance?**

- Define what "help" means in your DB tuning setup:
  - Better final performance: higher TPS / lower latency after N episodes.
  - Better sample efficiency: reach X% of best performance in fewer episodes.
  - Better safety: fewer DB crashes / fewer "obviously bad" configs.
    - Break hard rules
    - Resource blow-up: exceed the system memory / OS resources
  - Better validity: fewer rule violations (knob constraints broken).

RQ2: Are the benefits consistent across different DBMS (PostgreSQL vs. MySQL) and workloads (OLTP vs. OLAP)? For generalization.

RQ3: How do these rules impact the tuning performance?

- hard rules that impact DB crashes
- Soft rules that guide efficient tuning
- Implicit hardware resource bounded

RQ4: How much improvement can there would be when employ constraint-aware strategy to the existing state-of-the-art tuning method? How does the rule-augmented approach improve GPTuner (which uses LLM-guided BO) in terms of speed and final performance achieved? We evaluate the constraint-aware tuning of BO with GP and SMAC.

- BO-based methods: GPTuner [14],
- avoid local maximum
- speed up convergence
- avoid invalid configs

5.1 Experimental Setup

Workloads. Following the setup of GPTuner [14], we use TPC-H (OLAP type) with scale factor 1 and TPC-C (OLTP type) with factor 200 as our benchmarks. TPC-C uses ten terminals with unlimited arrival rate and the implementations are from BenchBase [5].

Hardware. The DBMS runs on top of the machine with a 12-core 12th Gen Intel(R) Core(TM) i7-12700K CPU, 32 GB of RAM and a 1 TB Samsung SSD.

Tuning Settings. We conduct all experiments using PostgreSQL v13. Referring to previous experiences [11, 14, 25], from the full configuration space, we select 45 tunable knobs that are relevant to performance-critical resource and dependency constraints (e.g., memory allocation, parallelism, and write-ahead logging behavior). We exclude parameters related to debugging, security, logging verbosity, and path configuration. The selected knobs are chosen to emphasize inter-parameter constraint structures, which are central to our study. For each workload, we perform four independent tuning runs with different random seeds to account for optimizer stochasticity. We report the mean best performance across all runs. Each tuning session consists of 200 iterations, among which the first 10 configurations are generated via LHS or topological sampling as the initial design. We optimize for overall system throughput (higher is better) for TPC-C and average latency (lower is better) for TPC-H [14].

Optimizers. We evaluate our approach using two widely adopted Bayesian optimization frameworks: SMAC and GP-BO. Both are shown to outperform alternative methods in most cases [33]. Both optimizers are implemented using the SMAC3 [19], ensuring a unified experimental infrastructure and eliminating implementation-level discrepancies. For SMAC, we use its default Random Forest surrogate model with expected improvement as the acquisition function. For GP-BO, we replace the surrogate model with a Gaussian Process while retaining the same acquisition optimization and evaluation protocol provided by the SMAC framework. This unified implementation ensures that differences in performance arise from the surrogate modeling strategy rather than engineering artifacts.

Evaluation Metrics. We use two metrics to evaluate the performance of CATune: final DBMS throughput (latency) improvement, and time-to-optimal throughput (latency). The former is related to the difference in performance (e.g., throughput, average latency) of the best-performing configurations found at the end of tuning (i.e., after 200 iterations).

5.2 Experimental Results: RQ1

Scenarios. **baseline**: the default search range for knobs without any constriats be awared. **rules only**: the default search range for knobs with extracted rules. **resource bounded only**: reducing search range due to limited hardware resources based on the default search range. **rules+bounded**: resource-bounded search range with rules.

Results in Tuning Performance and Sample Efficiency. Figure 5 illustrates the difference in final performance between different scenarios. Adding rules can get better final performance in 30 iterations less than baseline (i.e., default search range without rules). When we consider the resource limit and rules (bounded rules) for the search range, we can get better final performance in 75 iterations less than the baseline and rules. resource bounded only has a similar final performance to the baseline, but it has 10 iterations less than the baseline to reach the final performance. It finds a final performance of 44%+ on average, higher than the other three scenarios. Thus, the resource limit is critical for reducing the search range and saving tuning energy.

Results in Safety and Validity. From Figure 7, almost all successful iterations are from the exploitation stage, as the optimizer proposed. Only 1 success iteration in the default baseline group is from random search.

From Figure **??**, with iteration increases, the invalid configs are fewer and fewer. For exploitation, i.e., optimizer searches, we can see that adding rules and resource limits, there are no invalid configurations in the last 50 iterations, while another two experiments still have relative high possibility to propose invalid configurations. Thus, resource limit awareness is critical. Soft rules can reduce invalid configurations.

5.3 Experimental Results: Topological Sampling

6 DISCUSSION

6.1 Effect of Hallucinated Constraints

We study whether the The quality of constraints matters for the tuning performance.

6.2 Effect of Different Constraint Employment Strategies

Ordering knob dependency constraints can be employed in the BO-based method in different strategies. Here, we study how CATune is affected by two constraint employment strategies: (1) proactively filter out the bad infeasible or low-quality regions of the search space; (2) passively penalizes configurations that violate constraints during selection, without modifying the sampling process.

7 CONCLUSION

REFERENCES

[1] Jason Ansel, Shoaib Kamil, Kalyan Veeramachaneni, Jonathan Ragan-Kelley, Jeffrey Bosboom, Una-May O'Reilly, and Saman Amarasinghe. 2014. OpenTuner: an extensible framework for program autotuning. In *Proceedings of the 23rd International Conference on Parallel Architectures and Compilation* (Edmonton, AB, Canada) *(PACT '14)*. Association for Computing Machinery, New York, NY, USA, 303–316. https://doi.org/10.1145/2628071.2628092

[2] Baoqing Cai, Yu Liu, Ce Zhang, Guangyu Zhang, Ke Zhou, Li Liu, Chunhua Li, Bin Cheng, Jie Yang, and Jiashu Xing. 2022. HUNTER: An Online Cloud Database Hybrid Tuning System for Personalized Requirements. In *Proceedings of the 2022 International Conference on Management of Data* (Philadelphia, PA, USA) *(SIGMOD '22)*. Association for Computing Machinery, New York, NY, USA, 646–659. https://doi.org/10.1145/3514221.3517882

[3] Stefano Cereda, Stefano Valladares, Paolo Cremonesi, and Stefano Doni. 2021. CGPTuner: a contextual gaussian process bandit approach for the automatic

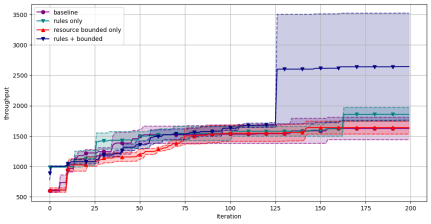

Figure 5: The results of final performance

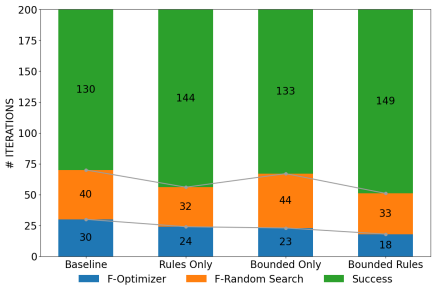

Figure 6: The number of successful iterations (Success, green) and the number of failed iterations in exploration (F-Random Search, blue) and exploitation (F-Optimizer, orange).

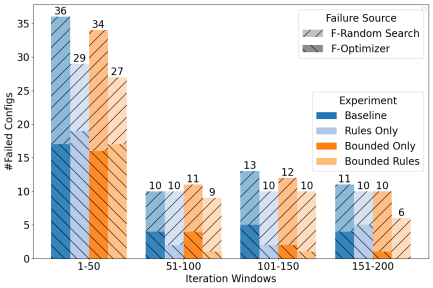

Figure 7: The number of invalid configurations every 50 iterations. The bars in the same color are for one experiment.

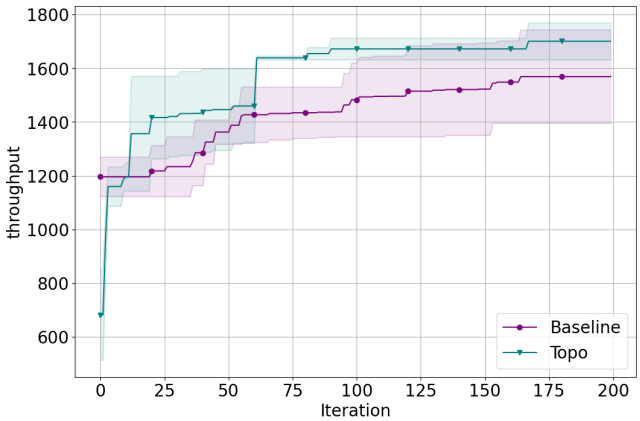

Figure 8: The results of using topological sampling on TPCC

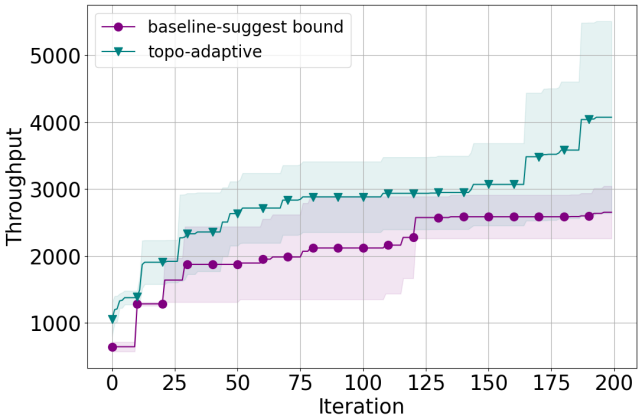

Figure 9: Applied Adaptive topological sampling with suggested bound on TPCC

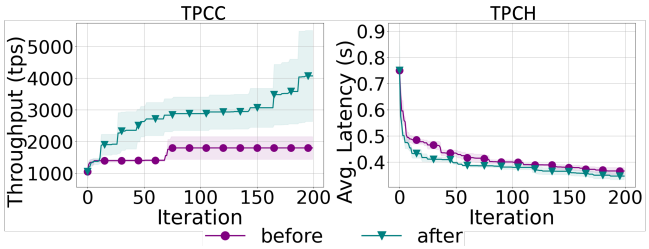

Figure 10: Effect of Hallucinated Constraints

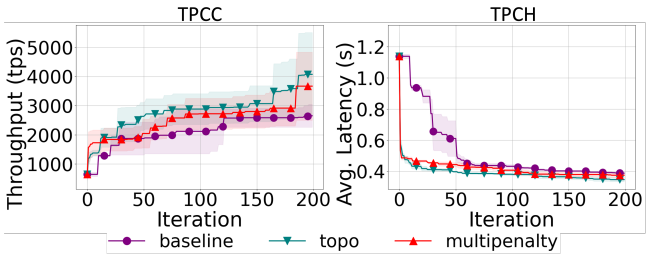

Figure 11: Effect of different sampling strategies.

tuning of IT configurations under varying workload conditions. *Proc. VLDB Endow.* 14, 8 (April 2021), 1401–1413. https://doi.org/10.14778/3457390.3457404

[4] Benoît Dageville and Mohamed Zait. 2002. SQL memory management in Oracle9i. In *Proceedings of the 28th International Conference on Very Large Data Bases* (Hong Kong, China) *(VLDB '02)*. VLDB Endowment, 962–973.

[5] Djellel Eddine Difallah, Andrew Pavlo, Carlo Curino, and Philippe Cudre-Mauroux. 2013. OLTP-Bench: an extensible testbed for benchmarking relational databases. *Proc. VLDB Endow.* 7, 4 (Dec. 2013), 277–288. https://doi.org/10.14778/2732240.2732246

[6] Songyun Duan, Vamsidhar Thummala, and Shivnath Babu. 2009. Tuning database configuration parameters with iTuned. *Proc. VLDB Endow.* 2, 1 (Aug. 2009), 1246–1257. https://doi.org/10.14778/1687627.1687767

[7] Jacob R. Gardner, Matt J. Kusner, Zhixiang Xu, Kilian Q. Weinberger, and John P. Cunningham. 2014. Bayesian optimization with inequality constraints. In *Proceedings of the 31st International Conference on International Conference on Machine Learning - Volume 32* (Beijing, China) *(ICML'14)*. JMLR.org, II–937–II–945.

[8] Michael A. Gelbart, Jasper Snoek, and Ryan P. Adams. 2014. Bayesian optimization with unknown constraints. In *Proceedings of the Thirtieth Conference on Uncertainty in Artificial Intelligence* (Quebec City, Quebec, Canada) *(UAI'14)*. AUAI Press, Arlington, Virginia, USA, 250–259.

[9] Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. 2018. Deep reinforcement learning that matters. In *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence and Thirtieth Innovative Applications of Artificial Intelligence Conference and Eighth AAAI Symposium on Educational Advances in Artificial Intelligence* (New Orleans, Louisiana, USA) *(AAAI'18/IAAI'18/EAAI'18)*. AAAI Press, Article 392, 8 pages.

[10] Konstantinos Kanellis, Ramnatthan Alagappan, and Shivaram Venkataraman. 2020. Too many knobs to tune? towards faster database tuning by pre-selecting important knobs. In *Proceedings of the 12th USENIX Conference on Hot Topics in Storage and File Systems (HotStorage '20)*. USENIX Association, USA, Article 16, 1 pages.

[11] Konstantinos Kanellis, Cong Ding, Brian Kroth, Andreas Müller, Carlo Curino, and Shivaram Venkataraman. 2022. LlamaTune: sample-efficient DBMS configuration tuning. *Proc. VLDB Endow.* 15, 11 (July 2022), 2953–2965. https://doi.org/10.14778/3551793.3551844

[12] Mayuresh Kunjir and Shivnath Babu. 2020. Black or White? How to Develop an AutoTuner for Memory-based Analytics. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data* (Portland, OR, USA) *(SIGMOD '20)*. Association for Computing Machinery, New York, NY, USA, 1667–1683. https://doi.org/10.1145/3318464.3380591

[13] Eva Kwan. 2002. Automatic Configuration for IBM ® DB 2 Universal Database TM Compressing years of performance tuning experience into seconds of execution. https://api.semanticscholar.org/CorpusID:15267980

[14] Jiale Lao, Yibo Wang, Yufei Li, Jianping Wang, Yunjia Zhang, Zhiyuan Cheng, Wanghu Chen, Mingjie Tang, and Jianguo Wang. 2024. GPTuner: A Manual-Reading Database Tuning System via GPT-Guided Bayesian Optimization. *Proc. VLDB Endow.* 17, 8 (April 2024), 1939–1952. https://doi.org/10.14778/3659437.3659449

[15] Jiale Lao, Yibo Wang, Yufei Li, Jianping Wang, Yunjia Zhang, Zhiyuan Cheng, Wanghu Chen, Mingjie Tang, and Jianguo Wang. 2025. GPTuner: An LLM-Based Database Tuning System. *SIGMOD Rec.* 54, 1 (April 2025), 101–110. https://doi.org/10.1145/3733620.3733641

[16] Jiale Lao, Yibo Wang, Yufei Li, Jianping Wang, Yunjia Zhang, Zhiyuan Cheng, Wanghu Chen, Yuanchun Zhou, Mingjie Tang, and Jianguo Wang. 2024. A Demonstration of GPTuner: A GPT-Based Manual-Reading Database Tuning System. In *Companion of the 2024 International Conference on Management of Data* (<conf-loc>, <city>Santiago AA</city>, <country>Chile</country>, </conf-loc>) *(SIGMOD/PODS '24)*. Association for Computing Machinery, New York, NY, USA, 504–507. https://doi.org/10.1145/3626246.3654739

[17] Cheng Li, Santu Rana, Sunil Gupta, Vu Nguyen, Svetha Venkatesh, Alessandra Sutti, David Rubin, Teo Slezak, Murray Height, Mazher Mohammed, and Ian Gibson. 2019. Accelerating Experimental Design by Incorporating Experimenter Hunches. arXiv:1907.09065 [stat.ML] https://arxiv.org/abs/1907.09065

[18] Guoliang Li, Xuanhe Zhou, Shifu Li, and Bo Gao. 2019. QTune: a query-aware database tuning system with deep reinforcement learning. *Proc. VLDB Endow.* 12, 12 (Aug. 2019), 2118–2130. https://doi.org/10.14778/3352063.3352129

[19] Marius Lindauer, Katharina Eggensperger, Matthias Feurer, André Biedenkapp, Difan Deng, Carolin Benjamins, Tim Ruhkopf, René Sass, and Frank Hutter. 2022. SMAC3: a versatile Bayesian optimization package for hyperparameter optimization. *J. Mach. Learn. Res.* 23, 1, Article 54 (Jan. 2022), 9 pages.

[20] Andrew Pavlo, Gustavo Angulo, Joy Arulraj, Haibin Lin, Jiexi Lin, Lin Ma, Prashanth Menon, Todd C. Mowry, Matthew Perron, Ian Quah, Siddharth Santurkar, Anthony Tomasic, Skye Toor, Dana Van Aken, Ziqi Wang, Yingjun Wu, Ran Xian, and Tieying Zhang. 2017. Self-Driving Database Management Systems. In *Conference on Innovative Data Systems Research*. https://api.semanticscholar.org/CorpusID:265531108

[21] Andrew Pavlo, Matthew Butrovich, Lin Ma, Prashanth Menon, Wan Shen Lim, Dana Van Aken, and William Zhang. 2021. Make your database system dream of electric sheep: towards self-driving operation. *Proc. VLDB Endow.* 14, 12 (July 2021), 3211–3221. https://doi.org/10.14778/3476311.3476411

[22] Victor Picheny. 2015. Multiobjective optimization using Gaussian process emulators via stepwise uncertainty reduction. *Statistics and Computing* 25, 6 (Nov. 2015), 1265–1280. https://doi.org/10.1007/s11222-014-9477-x

[23] Matthias Schonlau, William Welch, and Donald Jones. 1998. *Global versus local search in constrained optimization of computer models*. Vol. 34. 11–25. https://doi.org/10.1214/lnms/1215456182

[24] Jasper Snoek, Hugo Larochelle, and Ryan P Adams. 2012. Practical Bayesian Optimization of Machine Learning Algorithms. In *Advances in Neural Information Processing Systems*, F. Pereira, C.J. Burges, L. Bottou, and K.Q. Weinberger (Eds.), Vol. 25. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2012/file/05311655a15b75fab86956663e1819cd-Paper.pdf

[25] Immanuel Trummer. 2022. DB-BERT: A Database Tuning Tool that "Reads the Manual". In *Proceedings of the 2022 International Conference on Management of Data* (Philadelphia, PA, USA) *(SIGMOD '22)*. Association for Computing Machinery, New York, NY, USA, 190–203. https://doi.org/10.1145/3514221.3517843

[26] Immanuel Trummer. 2022. Demonstrating DB-BERT: A Database Tuning Tool that "Reads" the Manual. In *Proceedings of the 2022 International Conference on Management of Data* (Philadelphia, PA, USA) *(SIGMOD '22)*. Association for Computing Machinery, New York, NY, USA, 2437–2440. https://doi.org/10.1145/3514221.3520171

[27] Immanuel Trummer, Junxiong Wang, Ziyun Wei, Deepak Maram, Samuel Moseley, Saehan Jo, Joseph Antonakakis, and Ankush Rayabhari. 2021. SkinnerDB: Regret-bounded Query Evaluation via Reinforcement Learning. *ACM Trans. Database Syst.* 46, 3, Article 9 (Sept. 2021), 45 pages. https://doi.org/10.1145/3464389

[28] Dana Van Aken, Andrew Pavlo, Geoffrey J. Gordon, and Bohan Zhang. 2017. Automatic Database Management System Tuning Through Large-scale Machine Learning. In *Proceedings of the 2017 ACM International Conference on Management of Data* (Chicago, Illinois, USA) *(SIGMOD '17)*. Association for Computing Machinery, New York, NY, USA, 1009–1024. https://doi.org/10.1145/3035918.3064029

[29] Dana Van Aken, Dongsheng Yang, Sebastien Brillard, Ari Fiorino, Bohan Zhang, Christian Bilien, and Andrew Pavlo. 2021. An inquiry into machine learning-based automatic configuration tuning services on real-world database management systems. *Proc. VLDB Endow.* 14, 7 (March 2021), 1241–1253. https://doi.org/10.14778/3450980.3450992

[30] Junxiong Wang, Immanuel Trummer, and Debabrota Basu. 2021. UDO: universal database optimization using reinforcement learning. *Proc. VLDB Endow.* 14, 13 (Sept. 2021), 3402–3414. https://doi.org/10.14778/3484224.3484236

[31] Dani Yogatama and Gideon S. Mann. 2014. Efficient Transfer Learning Method for Automatic Hyperparameter Tuning. In *International Conference on Artificial Intelligence and Statistics*. https://api.semanticscholar.org/CorpusID:319311

[32] Ji Zhang, Yu Liu, Ke Zhou, Guoliang Li, Zhili Xiao, Bin Cheng, Jiashu Xing, Yangtao Wang, Tianheng Cheng, Li Liu, Minwei Ran, and Zekang Li. 2019. An End-to-End Automatic Cloud Database Tuning System Using Deep Reinforcement Learning. In *Proceedings of the 2019 International Conference on Management of Data* (Amsterdam, Netherlands) *(SIGMOD '19)*. Association for Computing Machinery, New York, NY, USA, 415–432. https://doi.org/10.1145/3299869.3300085

[33] Xinyi Zhang, Zhuo Chang, Yang Li, Hong Wu, Jian Tan, Feifei Li, and Bin Cui. 2022. Facilitating database tuning with hyper-parameter optimization: a comprehensive experimental evaluation. *Proc. VLDB Endow.* 15, 9 (May 2022), 1808–1821. https://doi.org/10.14778/3538598.3538604

[34] Xinyi Zhang, Hong Wu, Zhuo Chang, Shuowei Jin, Jian Tan, Feifei Li, Tieying Zhang, and Bin Cui. 2021. ResTune: Resource Oriented Tuning Boosted by Meta-Learning for Cloud Databases. In *Proceedings of the 2021 International Conference on Management of Data* (Virtual Event, China) *(SIGMOD '21)*. Association for Computing Machinery, New York, NY, USA, 2102–2114. https://doi.org/10.1145/3448016.3457291

[35] Xinyi Zhang, Hong Wu, Yang Li, Jian Tan, Feifei Li, and Bin Cui. 2022. Towards Dynamic and Safe Configuration Tuning for Cloud Databases. In *Proceedings of the 2022 International Conference on Management of Data* (Philadelphia, PA, USA) *(SIGMOD '22)*. Association for Computing Machinery, New York, NY, USA, 631–645. https://doi.org/10.1145/3514221.3526176

[36] Xinyi Zhang, Hong Wu, Yang Li, Zhengju Tang, Jian Tan, Feifei Li, and Bin Cui. 2023. An Efficient Transfer Learning Based Configuration Adviser for Database Tuning. *Proc. VLDB Endow.* 17, 3 (Nov. 2023), 539–552. https://doi.org/10.14778/3632093.3632114

[37] Yuqing Zhu, Jianxun Liu, Mengying Guo, Yungang Bao, Wenlong Ma, Zhuoyue Liu, Kunpeng Song, and Yingchun Yang. 2017. BestConfig: tapping the performance potential of systems via automatic configuration tuning. In *Proceedings of the 2017 Symposium on Cloud Computing* (Santa Clara, California) *(SoCC '17)*. Association for Computing Machinery, New York, NY, USA, 338–350. https://doi.org/10.1145/3127479.3128605